Cybersecurity by Design for Critical Infrastructures

Laurea Magistrale in Informatica

alma.pennacchi@uniroma2.it

Aprile 2026

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Roadmap

1 Transparent Middleware and Immutable Logs

Lezione 5 — Obiettivo

Domanda guida

Perché introdurre un middleware trasparente che osserva i flussi, registra i log e rende tali log verificabili?

- osservare il comportamento reale senza modificare la logica applicativa
- costruire log utili per detection, audit, SOC e forensic
- garantire integrità e non ripudio tramite meccanismi crittografici

Dove siamo nel corso

Finora abbiamo visto:

- baseline e deviazioni
- score e soglie
- detector rule-based, statistici, sequenziali e grafici
- costruzione di grafi e workflow dai log

Passo successivo

Serve una architettura che renda questi dati osservabili, persistenti e verificabili.

Il problema

In molte infrastrutture i flussi passano tra:

- frontend e backend
- servizi interni
- API esterne
- sistemi legacy
- componenti OT/cloud

Domanda

Come osserviamo questi flussi senza riscrivere tutti i sistemi esistenti?

Middleware trasparente

Definizione operativa

Un middleware trasparente è uno strato intermedio che riceve, inoltra e registra i flussi tra componenti, senza modificarne la semantica applicativa.

$$\text{Client} \rightarrow \text{Middleware} \rightarrow \text{Backend}$$

Idea

Il middleware diventa un punto naturale di osservazione, correlazione e logging.

Che cosa significa “trasparente”

Trasparente non significa invisibile o inutile.
Significa:

- non cambia il significato della richiesta
- inoltra input e output in modo controllato
- osserva metadati e comportamento
- registra eventi rilevanti

Messaggio

Il middleware non deve diventare la logica di business: deve essere un layer di osservabilità e controllo.

Cosa può osservare il middleware

Senza necessariamente leggere o conservare il payload completo, può registrare:

- timestamp
- sorgente e destinazione
- endpoint/API chiamata
- esito e codice risposta
- durata della richiesta
- dimensione input/output
- identità tecnica o applicativa

Log come eventi strutturati

Un log utile non dovrebbe essere solo testo libero.

Evento strutturato

$$e = (t,\ src,\ dst,\ action,\ outcome,\ metadata)$$

- t = tempo
- src, dst = sorgente e destinazione
- $action$ = operazione osservata
- $outcome$ = successo, errore, timeout, retry
- $metadata$ = contesto utile

Dal middleware al grafo dei flussi

Ogni richiesta osservata può generare:

- un nodo sorgente
- un nodo destinazione
- un arco orientato
- un timestamp
- un peso o attributo dell'arco

$$(src, dst, action, t) \Rightarrow src \xrightarrow{action} dst$$

Collegamento

Il middleware alimenta direttamente la costruzione del grafo comportamentale.

Correlare ciò che entra e ciò che esce

Il middleware osserva entrambi i lati:

- richiesta in ingresso
- richiesta inoltrata
- risposta del backend
- risposta restituita al client

Idea

La correlazione input/output permette di verificare se il workflow osservato è coerente.

Perché serve al SOC

Per un SOC, questi log permettono di:

- ricostruire timeline
- correlare eventi applicativi e di rete
- individuare deviazioni da baseline
- supportare triage e prioritizzazione
- fornire evidenza per incident response

Messaggio

Il middleware trasforma flussi applicativi in osservabili di sicurezza.

Perché serve all'audit

Per audit e compliance, i log devono rispondere a domande come:

- chi ha fatto cosa?
- quando?
- verso quale sistema?
- con quale esito?
- il log è stato alterato?

Punto critico

Un log utile per audit deve essere non solo leggibile, ma anche verificabile.

Il problema dell'integrità dei log

Se un attaccante compromette un sistema, può cercare di:

- cancellare log
- modificarli
- riscrivere timestamp
- rimuovere tracce dell'attacco

Conseguenza

La detection non basta: serve garantire che le evidenze non siano manipolabili.

Log immutabili

Idea

Un log immutabile è un log che, una volta registrato, non può essere modificato senza lasciare traccia.

Obiettivi:

- integrità
- non ripudio
- auditabilità
- supporto forensic

Hash dei log

Un primo passo è calcolare un hash dell'evento:

$$h_i = H(e_i)$$

dove:

- e_i = evento di log
- H = funzione hash crittografica
- h_i = impronta dell'evento

Proprietà

Se il log cambia, cambia anche il suo hash.

Hash chain

Per rendere evidente anche la cancellazione o modifica della sequenza:

$$h_i = H(e_i \parallel h_{i-1})$$

Ogni evento dipende dal precedente.

Conseguenza

Modificare un log passato rompe la catena degli hash successivi.

Perché usare blockchain

Idea

La blockchain può essere usata come registro distribuito per ancorare prove di integrità dei log.

Non serve necessariamente salvare tutto il payload:

- si possono salvare hash
- timestamp
- riferimenti agli eventi
- metadati minimi

Che cosa salva la blockchain

In un'architettura prudente, la blockchain non è il database dei log completi.
Può salvare:

- hash degli eventi
- hash di batch di log
- Merkle root
- timestamp di registrazione
- identità del produttore del log

Punto chiave

La blockchain certifica l'integrità; lo storage operativo conserva il dettaglio.

Architettura log + blockchain

$$\begin{aligned} \text{Middleware} &\rightarrow \text{Log Store} \\ &\rightarrow \text{Hash/Merkle Root} \\ &\rightarrow \text{Blockchain} \end{aligned}$$

- log completi nello storage operativo
- hash o root su blockchain
- verifica successiva confrontando log e proof

Merkle Root

Per evitare di scrivere ogni singolo log su blockchain, si raggruppano più eventi.

$$e_1, e_2, \ldots, e_n \rightarrow \text{Merkle Root}$$

Idea

La Merkle root rappresenta crittograficamente un intero batch di log.

Vantaggio

Riduce costo e complessità, mantenendo verificabilità.

Verifica di integrità

Per verificare un log:

1. recupero il log dallo storage
2. ricalcolo il suo hash
3. verifico appartenenza al batch
4. confronto la root con quella registrata su blockchain

Risultato

Se i valori coincidono, il log è coerente con quanto registrato al tempo dell'ancoraggio.

Perché non basta un database tradizionale

Un database tradizionale può essere:

- modificato da amministratori
- alterato da un attaccante con privilegi
- ripristinato da backup non coerenti
- difficile da usare come evidenza forte

Idea

La blockchain aggiunge un livello indipendente di verifica dell'integrità temporale.

Blockchain permissioned

In infrastrutture critiche ha senso una blockchain permissioned:

- partecipanti noti
- governance definita
- controllo degli accessi
- prestazioni più gestibili
- audit tra organizzazioni

Esempio concettuale

Org1 e Org2 mantengono un registro condiviso degli hash dei log critici.

Valore per il SOC

Per il SOC, log ancorati e verificabili migliorano:

- triage
- incident timeline
- correlazione tra eventi
- attribuzione tecnica
- prioritizzazione degli alert

Messaggio

Il SOC non lavora solo su eventi: lavora su evidenze affidabili.

Valore per forensic e incident response

In fase post-incidente serve dimostrare:

- quali eventi sono avvenuti
- in quale ordine
- se i log sono stati alterati
- quali workflow sono stati coinvolti
- dove si è propagato l'attacco

Idea

La blockchain non spiega da sola l'attacco, ma rafforza l'affidabilità delle evidenze.

Valore per audit e compliance

Log verificabili permettono di dimostrare:

- tracciabilità delle operazioni
- integrità dei registri
- separazione delle responsabilità
- rispetto di policy e workflow

Punto chiave

Auditability by design: la capacità di audit non viene aggiunta dopo, ma progettata nell'architettura.

Cosa NON risolve la blockchain

La blockchain non risolve automaticamente:

- qualità dei log
- correttezza della sorgente
- compromissione prima della registrazione
- interpretazione degli eventi
- detection e classificazione

Messaggio

Garbage in, immutable garbage out.

Rischio: falso senso di sicurezza

Un log immutabile non è automaticamente un log vero.
Possibili problemi:

- sorgente compromessa
- timestamp non affidabile
- campo mancante
- log incompleto
- evento già manipolato prima dell'hash

Conseguenza

Serve sicurezza della pipeline di logging, non solo blockchain.

Middleware come punto critico

Il middleware aumenta osservabilità, ma diventa anche un componente critico.
Deve essere progettato con:

- hardening
- autenticazione forte
- segregazione dei ruoli
- logging interno
- alta disponibilità

Privacy e minimizzazione

Non sempre è corretto salvare payload completi.
Principi:

- minimizzazione dei dati
- pseudonimizzazione dove possibile
- separazione tra log operativo e prova crittografica
- retention policy

Idea

Per audit spesso basta provare che un evento esisteva e non è stato alterato, senza esporre tutto il contenuto.

Architettura complessiva

$$
\begin{array}{ccccc}
\text{Client} & \rightarrow & \text{Middleware} & \rightarrow & \text{Backend} \\
 & \downarrow & & & \\
\text{Structured Logs} & \rightarrow & \text{Graph/Workflow Analysis} & \rightarrow & \text{SOC} \\
 & \downarrow & & & \\
\text{Hash/Merkle Root} & \rightarrow & \text{Blockchain} & &
\end{array}
$$

Dal flusso alla decisione

La catena completa diventa:

$$\begin{aligned} \text{Flow} &\rightarrow \text{Log} \\ &\rightarrow \text{Graph} \\ &\rightarrow \text{Detection} \\ &\rightarrow \text{Audit Proof} \end{aligned}$$

- il flusso genera osservazione
- il log struttura l'evento
- il grafo ricostruisce comportamento
- il detector produce alert
- la blockchain protegge l'evidenza

Messaggio finale

Idea centrale

Un middleware trasparente rende osservabili i flussi; la blockchain rende verificabile l'integrità delle evidenze.

Conclusione

In una infrastruttura critica, non basta rilevare un attacco: bisogna poter ricostruire, dimostrare e verificare ciò che è accaduto.